

CareerTailor — Architecture Review

Independent Technical Assessment | Production V1.1 | February 2026

Overall Verdict	Solid foundation with real engineering thinking. The RAG + hybrid storage approach is sound. However, several critical gaps need to be addressed before this is truly production-ready for the public.
------------------------	--

1. What You Got Right

Architecture & AI Design

- RAG over relational DB for semantic matching is the correct call. The V1.0→V1.1 upgrade reasoning is sound.
- Hybrid storage strategy (Pinecone + Supabase + Redis) is well thought out — right tool for each data type.
- LangGraph for orchestration is a strong choice for complex, stateful, multi-step agent workflows.
- Ephemeral resume file handling (.docx never stored server-side) is a genuinely good privacy design.
- Parallel scraping via Playwright for batch triage is architecturally correct.
- Gemini text-embedding-004 is a high-quality, cost-effective embedding model.
- Semantic Bridge (Phase B) — computing the UNION of missing skills and doing one combined interview is very user-friendly UX thinking.

2. Critical Issues (Must Fix Before Launch)

2.1 — Pinecone Is NOT Free at Scale

- Pinecone's free tier gives you 1 index and ~100K vectors. With public deployment and many users, you will hit this limit quickly and face real costs. Your stated goal is 'all free software' — Pinecone contradicts this.
- Replace Pinecone with pgvector (Postgres extension) on Supabase. Supabase's free tier supports pgvector natively. You already use Supabase, so this eliminates a paid dependency entirely and simplifies your stack to one database system. pgvector supports cosine similarity search and is production-proven.

2.2 — No Authentication / Multi-Tenancy Design

- The document mentions 'OAuth' in the diagram but there is no description of how user data is isolated. If user A's resume chunks get mixed with user B's in Pinecone/pgvector, you have a catastrophic privacy breach.

- Use Supabase Auth (free, built-in). Every vector upsert must include a `user_id` metadata field. Every query must filter by `user_id`. Enforce this with Supabase Row Level Security (RLS) policies.

2.3 — Web Scraping Is Legally and Technically Fragile

- Scraping LinkedIn, Indeed, Greenhouse, Lever, etc. violates their Terms of Service. LinkedIn actively blocks scrapers and will IP-ban your server. This is your most brittle component.

- Use official Job Posting APIs where available (LinkedIn has a partner API, Greenhouse/Lever/Workday have public JSON endpoints). For sites without APIs, use a scraping proxy service like ScraperAPI (has a free tier) or implement respectful rate limiting + user-agent rotation with explicit robots.txt compliance. Be prepared for this to break regularly.

2.4 — No Error Handling / Resilience Strategy

- LangGraph node failures, Playwright timeouts, Gemini API rate limits, and network errors will all crash the pipeline. There is zero mention of retry logic, fallback behavior, or dead-letter queues.

- Implement retry with exponential backoff for all LLM and external API calls. Use LangGraph's built-in error handling to catch node failures and surface them to the user gracefully. Add a job status table in Supabase (pending / running / failed / complete) so users can see what happened.

2.5 — Redis Is Listed But Not Justified

- Redis for 24-hour TTL cache of scraped job text adds operational complexity. For a public free deployment, running Redis means another service to manage/pay for.

- Store scraped job text directly in a Supabase table with a `created_at` timestamp and run a Supabase cron job (`pg_cron`, free) to purge rows older than 24 hours. Eliminates Redis entirely.

3. Significant Gaps (Important for Production Quality)

3.1 — No Rate Limiting or Abuse Protection

- A public deployment with no rate limiting means one bad actor can exhaust your Gemini API quota (the only thing you're paying for) in minutes.

- Implement per-user rate limiting at the FastAPI layer using `slowapi` (free, Redis or in-memory). Limit: e.g., 5 job scans and 3 resume generations per user per day on the free tier.

3.2 — Playwright in Production Is Heavy

- Running Playwright (Chromium) in a server environment is memory-intensive (~300-500MB per browser instance). On free hosting tiers (Render, Railway, Fly.io free tier), this will cause OOM crashes.

- Use Playwright only as a fallback. First attempt `httpx` + `BeautifulSoup` (lightweight, fast). Only launch Playwright if the JS-rendered DOM is needed. Alternatively, use a dedicated scraping microservice deployed separately.

3.3 — The .docx Injection Strategy Needs Detail

■ ■ 'Intelligently inject keywords into specific paragraph runs (preserving formatting)' is the hardest problem in this entire system. Word documents are complex XML. Naive string injection breaks formatting, styles, and tracked changes.

■ Use python-docx carefully — operate at the Run level, not Paragraph level. Never use string.replace() on raw XML. Consider a template-based approach: maintain a structured JSON representation of the resume and re-render the .docx from that JSON using a Jinja2 template. This gives you reliable, testable output.

3.4 — No Deployment Architecture Described

■ ■ The document describes application logic but not where/how this runs publicly. FastAPI + Playwright + LangGraph is not a simple deployment.

■ For a zero-cost public deployment: host FastAPI on Render (free tier, with sleep) or Railway (free tier). Host the React/Vite frontend on Vercel or Netlify (free). Use Supabase free tier for all data. Use Supabase Edge Functions for lightweight operations. Document this explicitly so contributors can replicate it.

3.5 — Google Drive Integration Has Auth Complexity

■ ■ GDrive OAuth requires a verified app for public deployment. Google's verification process can take weeks and requires a privacy policy, domain verification, and security review.

■ Make GDrive optional (a premium feature). Default flow should be local download. This unblocks your MVP launch significantly.

4. Recommendations to Reach Enterprise Quality

4.1 — Observability (You're Flying Blind Without This)

Enterprise systems require visibility into what the AI is doing. Add LangSmith (free tier available) for LangGraph tracing — it shows you exactly which nodes ran, what prompts were sent, and where failures occurred. Add Sentry (free tier) for error tracking in FastAPI.

4.2 — Prompt Versioning

Your Gemini prompts for Feasibility Scoring and keyword injection are core business logic. Store them in a prompts/ directory with version numbers, not hardcoded in Python files. This lets you A/B test prompts and roll back bad changes. LangSmith also supports prompt management.

4.3 — Output Quality Validation

Before streaming a generated resume to the user, run a simple validation pass: Did the injected keywords actually appear in the output? Is the word count within $\pm 15\%$ of the original? Did the LLM accidentally remove sections? Fail loudly and retry rather than delivering a broken resume.

4.4 — Async Everything

FastAPI is async-native. Make sure all your LangGraph nodes, database calls (use `asyncpg` or Supabase async client), and HTTP requests (use `httpx async`) are truly async. Synchronous blocking calls inside an async FastAPI app will kill your throughput under any real user load.

4.5 — Consider a Queue for Long-Running Jobs

Scraping 5 jobs + running LangGraph + generating a resume can take 30-90 seconds. HTTP connections time out. Instead of a synchronous API call, use a job queue pattern: `POST /jobs` returns a `job_id` immediately, a background worker processes it, and the frontend polls `GET /jobs/{id}/status`. Use Supabase Realtime (free) to push status updates.

4.6 — Document Your Prompt Engineering

The Feasibility Score (0-100%) is a key feature. Document what criteria the prompt uses to compute it. What makes a job Red vs Yellow vs Green? If this is just a vibes-based LLM call, it will be inconsistent. Define the rubric explicitly in the prompt and in your docs.

5. Revised Free Stack Recommendation

Component	Original	Recommended (Free)	Reason
Vector DB	Pinecone	pgvector on Supabase	Free, already in stack, no extra service
Cache	Redis	Supabase table + <code>pg_cron</code>	Eliminates Redis entirely
Auth	Unspecified	Supabase Auth + RLS	Free, built-in, handles multi-tenancy
Frontend Host	Unspecified	Vercel / Netlify	Free tier, CDN, CI/CD included
Backend Host	Unspecified	Render / Railway	Free tier for FastAPI
Error Tracking	None	Sentry (free tier)	Essential for production
LLM Tracing	None	LangSmith (free tier)	Debug LangGraph workflows
Rate Limiting	None	slowapi	Protect your paid Gemini quota

6. Prioritized Action Plan

Phase 1 — Before Any Public Launch (Critical):

1. Replace Pinecone with pgvector on Supabase
2. Implement Supabase Auth + RLS for multi-tenant data isolation
3. Add per-user rate limiting to protect Gemini quota
4. Add job status tracking (async queue pattern) to handle long-running workflows
5. Add retry/error handling in all LangGraph nodes

Phase 2 — Before Scale (Important):

6. Add LangSmith tracing + Sentry error monitoring
7. Replace Redis with Supabase table + pg_cron
8. Make Google Drive optional (local download as default)
9. Harden .docx injection with run-level python-docx operations

Phase 3 — Quality & Growth:

10. Add output validation before delivering generated resumes
11. Version and document all LLM prompts
12. Document deployment architecture for contributors
13. Define explicit rubric for Feasibility Score computation

This review was generated by Claude (Anthropic) based on the CareerTailor System Architecture Document V1.1 — February 2026. All recommendations are opinionated engineering guidance and should be validated against your specific constraints and usage patterns.

SECTION 02 - ARCHITECTURE DIAGRAMS

System Architecture Diagrams

High-Level | Data Flow | Storage Strategy | Component Map

02

● PRODUCTION V1.1 — REVISED ARCHITECTURE

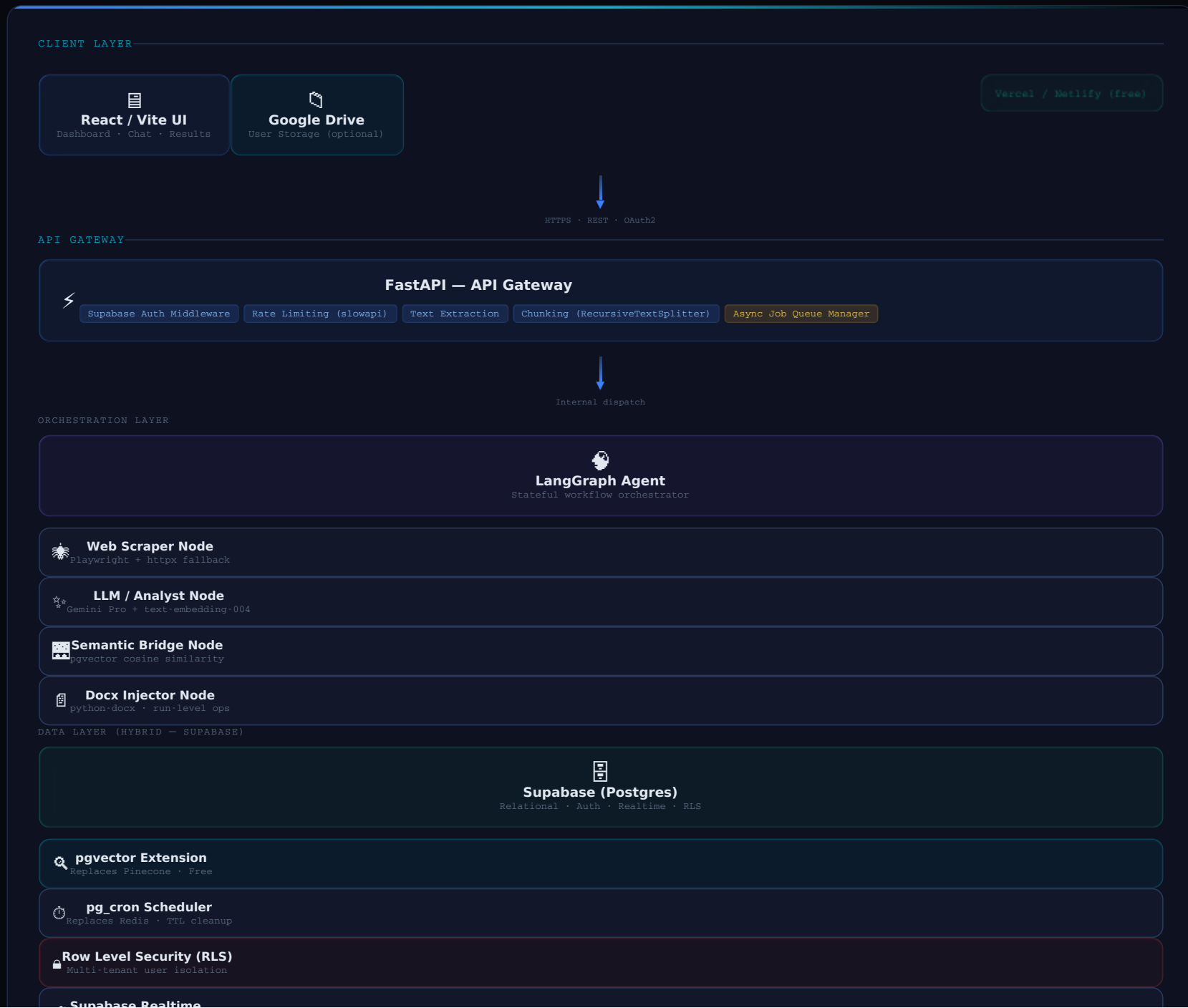
CareerTailor

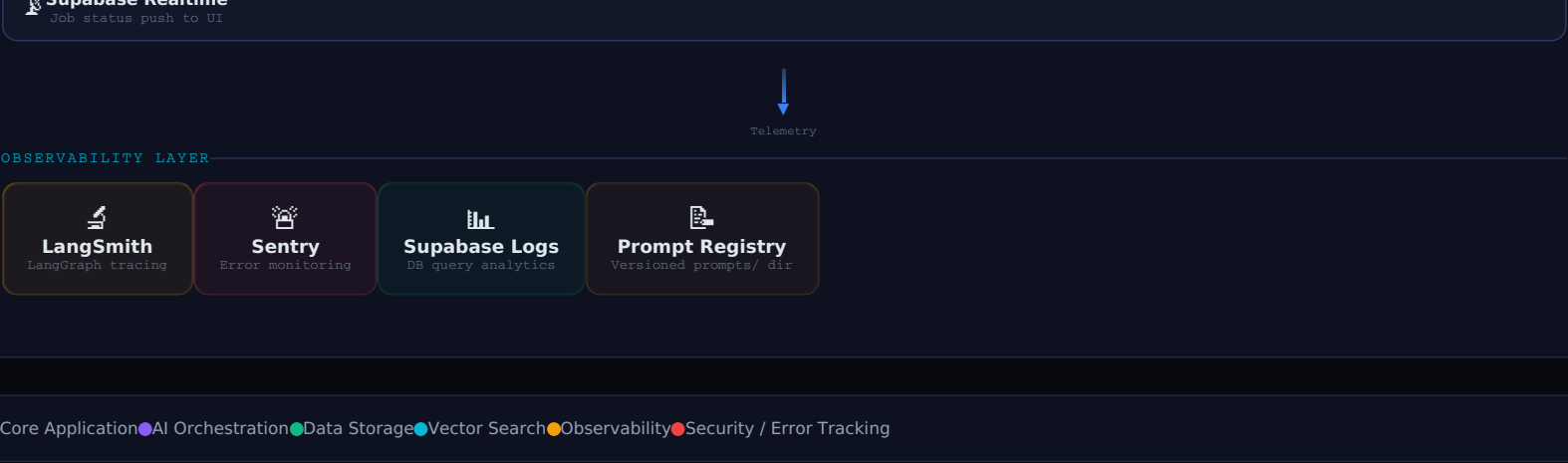
System Architecture Diagrams

VERSION	DATE	STATUS	STACK
1.1 Revised	February 2026	Approved	All Free Tier

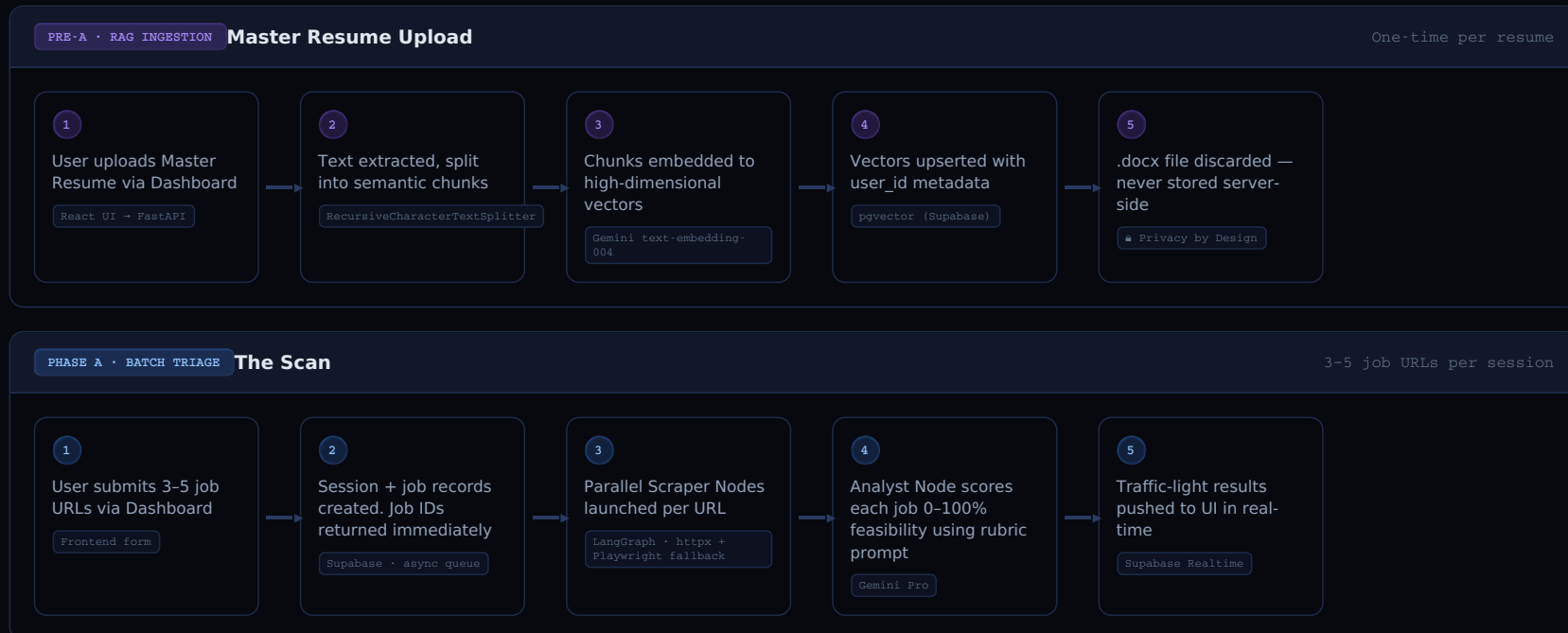
DIAGRAMS INCLUDED

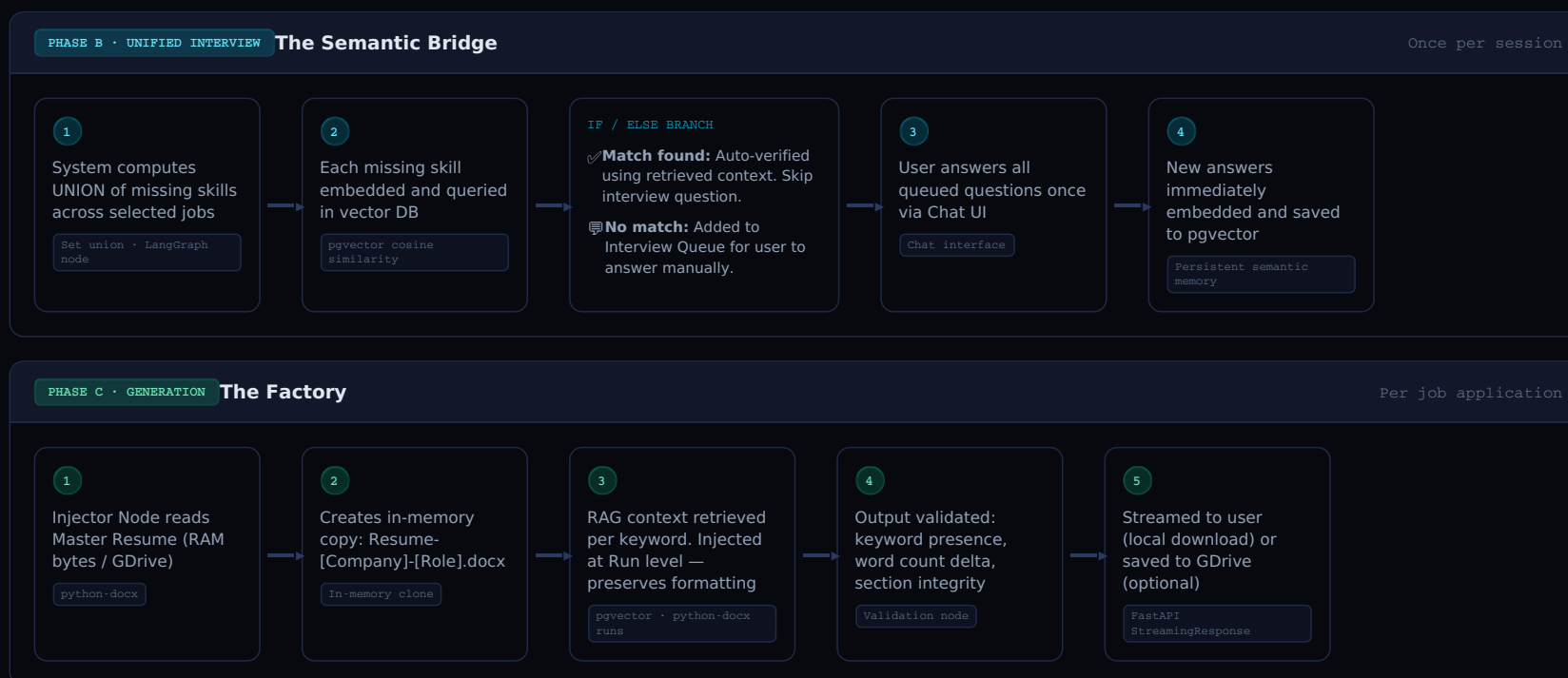
- 01 High-Level System Architecture
- 02 End-to-End Data Flow (4 Phases)
- 03 Hybrid Storage Strategy
- 04 Component Dependency Map





Flow Overview: A user session passes through 4 sequential phases. Each phase is independently retryable. Job status persisted in Supabase allows users to resume interrupted sessions. The async queue pattern (POST returns job_id instantly) prevents HTTP timeout on long-running AI operations.





Design Principle: Each data type is stored in the system best suited for it. All storage runs within Supabase free tier, eliminating paid external services (Pinecone, Redis). The only paid service in the entire stack is the Gemini API for AI generation.

 **pgvector**

Supabase Extension · Vector DB · Replaces Pinecone

Retention	Persisted
Dimensions	768-d (Gemini embeds)
Index	ivfflat (cosine)
Isolation	RLS · user_id filter

Stores resume chunk embeddings and interview answer embeddings. Enables cosine similarity search for the Semantic Bridge phase.

Resume Chunks

Answer Embeddings

 **Supabase Postgres**

Relational · Auth · Realtime · RLS

Retention	Persisted
Key tables	sessions, jobs, users
Auth	Supabase Auth built-in
Realtime	Job status push events

Session state, job queue status, UI history sidebar, app state, audit logs. Source of truth for all relational / structured data.

Sessions

Job Queue

Audit Log

 **Supabase Table + pg_cron**

TTL Cache · Replaces Redis entirely

Retention	24-hour TTL
Cleanup	pg_cron hourly job
Extra cost	\$0 (no Redis needed)

Temporary storage for scraped job text during parallel processing. Automatically purged via Postgres scheduled job — no separate Redis service required.

Scraped Job Text

Dedup Cache

 **RAM / Google Drive**

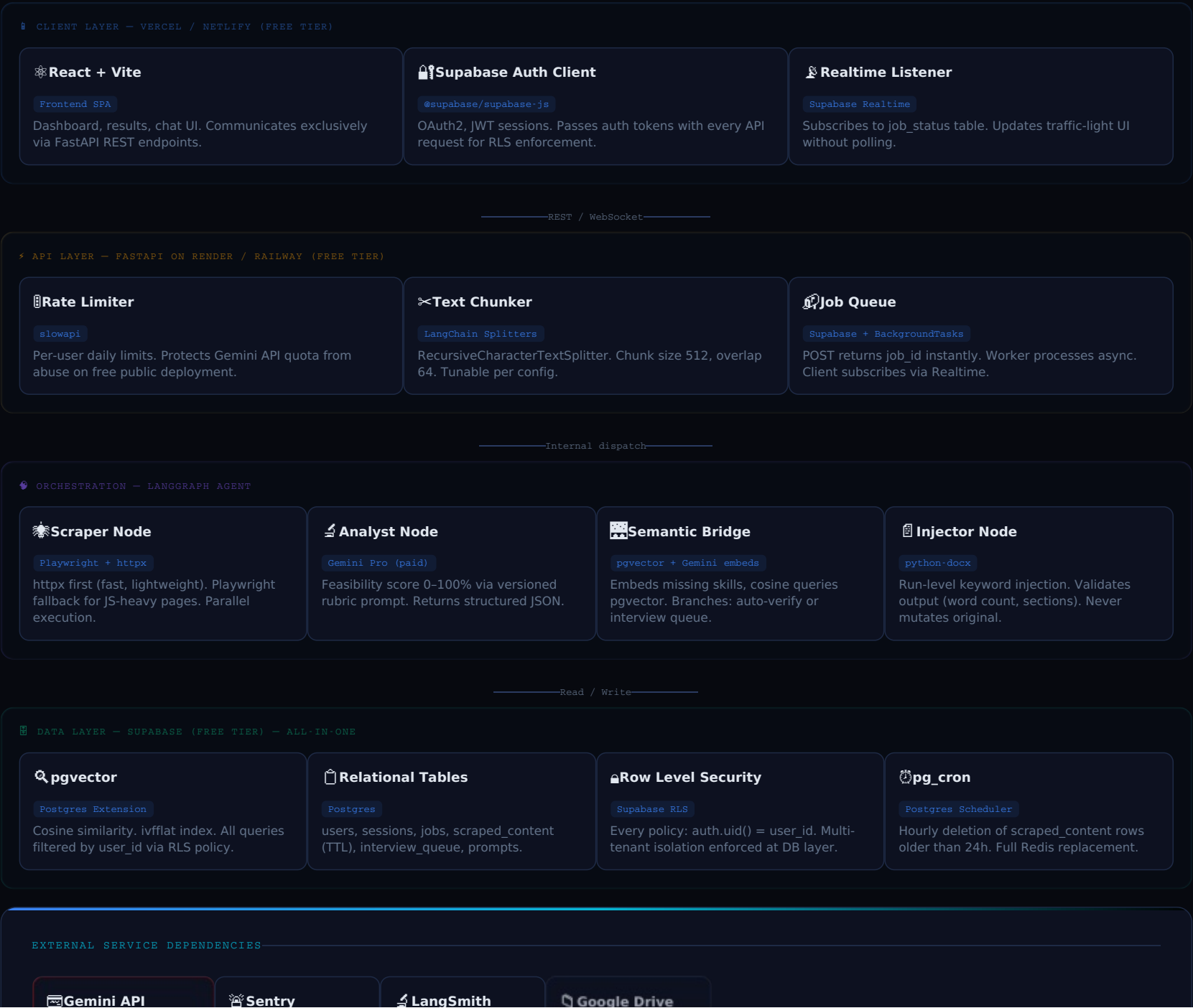
Ephemeral · User-Owned · Privacy by Design

Retention	Ephemeral (RAM only)
Server copy	None — never stored
Ownership	100% user-controlled

Resume .docx files live only in-memory during generation. Output is streamed directly to user or saved to their own GDrive. Zero server-side file storage.

DATA OWNERSHIP MATRIX

Data Type	Storage Location	Who Owns It	User Can Delete?	Shared with 3rd Party?
Resume File (.docx)	RAM (ephemeral)	User	N/A — never stored	No
Resume Embeddings	pgvector (Supabase)	Platform (scoped to user)	Yes — on account deletion	Gemini API (embedding call)
Session / Job History	Supabase Postgres	Platform (scoped to user)	Yes	No
Scraped Job Text	Supabase table (24h TTL)	Ephemeral	Auto-expired	No



PAID — Only Cost

Pro (generation) + text-embedding-004.
Protected by slowapi
rate limiting.

Free tier

Error tracking for
FastAPI + React. 5K
errors/mo free.

Free tier

LangGraph trace
visibility + prompt
version management.

Optional feature

Output save only.
Launch as optional —
requires OAuth app
verification.